

Kitana 加密演算法規格白皮書

基於 Rust 密碼學架構的動態流式防禦系統與跨平台相容實作

專案：Kitana Cryptool Core • 版本：v1.0 • 安全層級：高 (AES-256-CBC)

1. 概述

本白皮書旨在規範與說明 **Kitana Cryptool** 所採用的核心加密架構與二進位檔案結構（以下簡稱「Kitana 式加密法」）。本系統採用純 Rust 密碼學組件（Pure Rust Crypto）建構，捨棄了對系統底層動態連結庫（如 OpenSSL）的依賴，以確保在 Windows、Linux 與 macOS 等不同作業系統平台上，皆具備 100% 一致的金鑰衍生結果與加解密行為。

Kitana 式加密法專為大檔案處理設計，採用嚴格的「流式處理（Streaming）」控制與「安全水庫累積機制」，在極低的記憶體腳印下，提供具備進度回調（Progress Callback）與動態取消（Cancel Flag）的高安全級防禦實作。

2. 密碼學核心參數

Kitana 式加密法嚴格遵循現代工業標準密碼學演算法，其核心配置參數如下表所示：

參數名稱	規格與演算法	長度 / 疊代次數	安全用途
對稱加密演算法	AES-256-CBC 模式	—	高強度區塊資料加密
金鑰衍生函數 (KDF)	PBKDF2-HMAC-SHA256	100,000 次疊代	防止字典攻擊與密碼暴力破解
隨機鹽值 (Salt)	密碼 secure 隨機數生成	16 位元組 (128 bits)	預防彩虹表攻擊，使同密碼產生不同密文
主金鑰 (Key)	AES-256 Key	32 位元組 (256 bits)	AES 晶片核心加密金鑰
初始化向量 (IV)	CBC 鏈結模式向量	16 位元組 (128 bits)	確保相同的明文區塊加密後產生不同密文
資料填充模式	PKCS#7 Padding	對齊 16 位元組區塊	處理非 16 位元組倍數的檔案末尾資料

3. 檔案二進位結構 (Binary Layout)

經由 Kitana 加密法產生的密文檔案，具備結構高度緊湊、無冗餘標頭的特點。其磁碟二進位布局如下：

$$\text{File_Encrypted} = \text{Salt_16_bytes} \parallel \text{Ciphertext_N_bytes}$$

- **Salt (前 16 位元組)**：每個加密檔案獨一無二的隨機鹽值。解密時必須首先讀取此 16 位元組以啟動金鑰衍生。
- **Ciphertext (後續所有資料)**：經由 AES-256-CBC 加密後的密文主體，其總長度必然為 16 位元組的整數倍（在標準狀況下）。

4. 加密流程與核心演算法

4.1 金鑰衍生階段

系統接收使用者輸入的明文密碼 P 。首先生成 16 位元組的隨機數 S 。利用 PBKDF2 演算法衍生出長度為 48 位元組的混合緩衝區，並切割為 Key 與 IV：

$$\begin{aligned} \text{Key_IV_48_bytes} &= \text{PBKDF2}(P, S, c=100000, dkLen=48) \\ \text{Key} &= \text{Key_IV_0..32}, \text{IV} = \text{Key_IV_32..48} \end{aligned}$$

4.2 流式加密循環 (4096 輸送帶)

為了兼顧記憶體效能，加密採用串流動態傳送機制：

1. 配置大小為 4096 位元組的固定緩衝區（輸送帶）。
2. 每次迴圈自輸入流讀取最多 4096 位元組資料。
3. **中途檢查**：若檢測到取消旗標（Cancel Flag），立刻中斷並回傳操作取消。
4. **區塊加密**：若讀滿 4096 位元組，直接以 16 位元組為單位執行原地（In-place）區塊加密並寫出。
5. **進度回調**：更新已處理位元組數，換算百分比觸發進度調度器。

4.3 邊界與 Padding 處理

當檔案讀取至末端，剩餘資料長度 $L_{\text{last}} < 4096$ 時，啟動 PKCS#7 填充：

- 計算需填充的位元組數： $\text{Padding} = 16 - (L_{\text{last}} \bmod 16)$ 。
- 在資料尾端補上數量為 Padding 、數值亦為 Padding 的位元組。
- 通過加密器寫出最後的完整區塊。若原始檔案剛好對齊，則追加一個包含 16 位元組 0×10 的獨立 Padding 區塊。

5. 解密流程與超級相容機制 (水庫防線)

由於歷史上其他語言（如部分 C++ 舊實作版）在文件尾端寫入時，可能因底層 I/O 迴圈邊界判定錯誤而多寫入 1~15 位元組的非對齊垃圾碎屑，Kitana 解密法特別設計了「**累積水庫與動態裁剪機制**」，具備極高的容錯能力。

5.1 累積水庫 (Ciphertext Accumulator)

解密流讀取密文時，並不直接對單一區塊進行解填充，而是將資料流注入一個動態向量水庫。

水庫防線核心邏輯

水庫內永遠扣留最後至少 16 位元組的資料不急於解密，防止誤將仍含有 Padding 的最後一個區塊當作普通資料區塊解密寫出。

5.2 尾部垃圾動態裁剪 (Truncate 機制)

當密文流完全讀取完畢，跳出主迴圈後，水庫中留下了檔案最末端的關鍵區塊。此時 Kitana 啟動防禦檢查：

1. 計算當前水庫剩餘密文總長度 $Len_{\{rem\}}$ 。
2. 計算非對齊餘數： $Remainder = Len_{\{rem\}} \bmod \{16\}$ 。
3. **強行裁剪**：若 $Remainder > 0$ ，判定此餘數為舊版環境多寫入的非標準垃圾碎屑，程式自動執行：

```
Valid_Len = Len_{rem} - Remainder
ciphertext_accumulator.truncate(Valid_Len)
```

直接將不滿 16 位元組的無效尾巴削去。

4. 對裁剪後 100% 對齊的標準密文塊執行反向 PKCS#7 解填充 (Unpadding)，安全還原文文。

6. 安全性與設計評估

- **無狀態穩定性**：由於不依賴作業系統 OpenSSL 機制，此 Rust 核心實作的金鑰衍生與置換表不受底層 OS 更新影響，具有永久的可還原性。
- **抗密碼分析能力**：100,000 次疊代的 PBKDF2 大幅提高了攻擊者的時間成本，能有效防禦基於 GPU 的現代化撞庫與彩虹表離線攻擊。
- **高可用性設計**：雙向進度條控制與防爆記憶體流設計，使 Kitana 加密法在面對數十 GB 等級的極端大檔案時，依然能保持作業系統 UI 的流暢回應與記憶體低消耗。